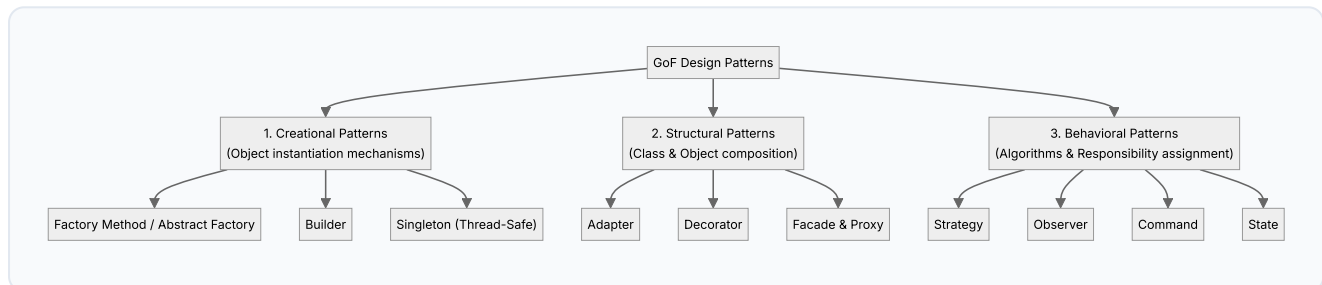


LLD Chapter 3: The Complete Design Patterns Catalog in Modern Python

Core Learning Objective: Master the Gang of Four (GoF) Design Patterns implemented with modern Python 3.12+ idioms. Understand the exact problem each pattern solves, its UML architecture, and real-world enterprise implementations.

1. Categorization Matrix of GoF Patterns



2. Creational Patterns

1. Factory Method & Abstract Factory

- **Problem:** Decouple object creation logic from client code so new variants can be added without modifying existing factory consumers.

2. Builder Pattern

- **Problem:** Constructing complex composite objects with dozens of optional configurations without constructor telescoping.

3. Structural Patterns

1. Adapter Pattern

- **Problem:** Convert the interface of an incompatible third-party or legacy class into another interface clients expect.



2. Proxy Pattern

- **Problem:** Provide a surrogate or placeholder for another object to control access, perform lazy initialization, or cache results.



4. Behavioral Patterns

1. Strategy Pattern

- **Problem:** Define a family of interchangeable algorithms, encapsulate each one, and make them swappable at runtime.



2. Observer Pattern (Pub-Sub)

- **Problem:** Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified automatically.



3. Command Pattern

- **Problem:** Encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations.



4. State Pattern

- **Problem:** Allow an object to alter its behavior when its internal state changes. The object will appear to change its class.

Further Reading

- [Refactoring Guru: pattern catalog](#)
 - [Wikipedia: Design Patterns](#)
 - [Python typing.Protocol](#)
-

Check Yourself

Answer in your head or on paper first, then open each answer.

▼ 1. Strategy versus State: how do they differ although the structure looks alike?

Strategy lets the client choose an interchangeable algorithm; State lets an object change behaviour automatically as its internal state changes.

▼ 2. When is Singleton a bad idea?

It hides global state, hurts testing and concurrency; prefer dependency injection of a single instance.

▼ 3. What problem does Observer solve?

Notifying many dependents of changes without the subject knowing who they are (event handlers, pub/sub).

▼ 4. What does Factory give you over `new`/constructors?

Centralised creation logic and the ability to return different concrete types behind one interface.